DMA Controller

IO & Signal Reference Document

AHB/APB Bus System · 32-bit RISC-V · Verilog RTL

Module	`dma_controller`
Bus	APB (config) + AHB (data)
Language	Verilog (IEEE 1364-2005)
Status	Verified via simulation

This document covers all port signals, internal registers, FSM states, transfer workflow, and bug fixes identified during simulation.

Contents

1. Overview

A **Direct Memory Access (DMA)** controller moves blocks of data between memory and peripherals without keeping the CPU occupied for every byte.

The CPU programs the DMA *once* (five register writes over APB), sets the enable bit, then returns to executing code. The DMA handles bus arbitration, read cycles, write cycles, address incrementing, and beat counting autonomously. It interrupts the CPU only when the entire block is done.

Three separate conversations happen in every DMA transfer:

1. APB	CPU → DMA config registers (slow, register access only)
2. AHB	DMA ↔ Memory / Peripherals (fast, bulk data)
3. Sideband	DMA ↔ Peripheral handshake + IRQ to CPU

1.1. Module Signature

```
module dma_controller(
    input  wire        PCLK, HCLK,         // APB clock, AHB clock
    input  wire        PRESETN, HRESETN,   // Active-low resets
    // APB Slave interface
    input  wire        PSEL, PENABLE, PWRITE,
    input  wire [31:0] PADDR, PWDATA,
    output reg  [31:0] PRDATA,
    output wire        PREADY,
    // AHB Master interface
    input  wire        HGRANT, HREADY,
    input  wire [31:0] HRDATA,
    output reg         HBUSREQ, HWRITE, irq,
    output reg  [31:0] HADDR, HWDATA,
    output reg  [1:0]  HTRANS
);
```

2. Port Signal Reference

2.1. Global Signals

Why active-low reset? The AHB/APB specification mandates active-low resets. The sensitivity list `negedge HRESETN` means reset fires on the *falling* edge — asynchronously, without waiting for the next clock. This guarantees the system reaches a safe state immediately at power-on.

2.2. APB Slave Interface — Configuration Port

The CPU uses APB to write the DMA's internal configuration registers before a transfer begins. APB is a simple two-cycle protocol — not used for bulk data.

2.2.1. APB 2-Cycle Protocol — Timing Detail

Every APB access takes **exactly two clock cycles**:

Cycle 1 (Setup): CPU drives PSEL=1, PWRITE, PADDR, PWDATA. PENABLE=0.
Cycle 2 (Enable): CPU raises PENABLE=1. DMA captures PWDATA on this rising edge.

The two-cycle design gives the slave a full cycle to prepare, eliminating glitches and setup-time violations that would occur if data was captured in the same cycle it was placed on the bus.

Table 1: Global / Clock / Reset Signals

lightblue **Signal**	**Dir**	**Width**	**Description**
PCLK	in	1-bit	APB clock. All APB register writes/reads are sampled on its rising edge. Typically slower than HCLK in real systems.
HCLK	in	1-bit	AHB clock. All FSM transitions and AHB signal changes are synchronised to this. In this design both clocks are tied to the same frequency.
PRESETN	in	1-bit	APB active-low reset. When LOW, all APB config registers (SRC, DST, LEN, CTRL) are cleared to 0.
HRESETN	in	1-bit	AHB active-low reset. When LOW, FSM goes to IDLE, all working pointers and IRQ are cleared.

```
// APB write fires only when all three are high simultaneously
always @(posedge PCLK or negedge PRESETN) begin
    if (!PRESETN) begin
        ...reset registers...
    end else if (PSEL && PENABLE && PWRITE) begin
        case (PADDR[7:0])
            8'h00: src_addr_reg <= PWDATA;
            ...
        endcase
    end
end
```

2.3. AHB Master Interface — Data Movement Port

The DMA acts as an AHB **master**. It drives addresses and controls transfers. Memory and peripherals are AHB **slaves** — they respond to the DMA. The DMA must request and win bus arbitration before driving any signal.

Why is HWDATA combinational?
HWDATA is driven as `always @(*) HWDATA = data_buf`. If registered (`HWDATA <= data_buf` on clock edge), HWDATA updates one cycle *after* entering the write state — but the memory slave captures it *in* that first cycle. The slave would always see stale data (one beat behind). Combinational assignment ensures HWDATA is valid the instant `data_buf` is stable, which is the cycle *after* it was latched from HRDATA.

2.4. Sideband / Interrupt Signal

3. Internal Register Map

3.1. APB-Visible Configuration Registers

These are flip-flop registers **inside the DMA chip**. The CPU writes them over APB before starting a transfer. They are the DMA's "to-do list."

Table 2: APB Slave Interface Signals

lightblue **Signal**	**Dir**	**Width**	**Description**
PSEL	in	1-bit	Chip-select for this DMA slave. CPU asserts high to say "I am talking to THIS slave." Without it the DMA ignores all other APB signals.
PENABLE	in	1-bit	Two-cycle enable strobe. Cycle 1: PENABLE=0 (setup). Cycle 2: PENABLE=1 (enable). DMA latches PWDATA only when both PSEL=1 and PENABLE=1 and PWRITE=1 are simultaneously true.
PWRITE	in	1-bit	Direction: 1 = CPU writing to DMA register. 0 = CPU reading from DMA register (PRDATA driven back).
PADDR	in	32-bit	Register offset. Only bits [7:0] are decoded. Selects which internal register to access: 0x00=SRC, 0x04=DST, 0x08=LEN, 0x0C=CTRL, 0x10=STATUS, 0x14=INT_CLR.
PWDATA	in	32-bit	Write data from CPU to the selected register. Valid when PSEL & PENABLE & PWRITE are all high.
PRDATA	out	32-bit	Read data from DMA to CPU. CPU reads this to check STATUS (BUSY/DONE/ERR) or verify programmed values. Driven combinationally from the selected register.
PREADY	out	1-bit	Slave stall signal. 1 = DMA has finished the current access. In this implementation permanently tied to 1 (no stalling). In full designs, held low if DMA needs extra decode cycles.

3.2. Internal Working Registers (not APB-visible)

These are flip-flops inside the DMA datapath, used during an active transfer.

3.2.1. Byte Address vs. Word Index — Why HADDR[7:2]

Memory in hardware is byte-addressed: every address points to 1 byte.
HADDR is a byte address — it increments by 1 per byte.

The Verilog memory array `reg [31:0] mem [0:63]` is **word-indexed**: each slot holds 32 bits (4 bytes). Its index is a word number, not a byte address.

Conversion: word index = byte address ÷ 4

Dividing by 4 in binary = dropping the bottom 2 bits:

`mem[HADDR[7:2]]` ≡ `mem[HADDR » 2]` ≡ `mem[HADDR / 4]`

Table 3: AHB Master Interface Signals

lightblue **Signal**	**Dir**	**Width**	**Description**
HBUSREQ	out	1-bit	DMA requests ownership of the AHB bus. Held HIGH throughout the entire transfer. DMA must NOT drive HADDR or HTRANS until HGRANT=1.
HGRANT	in	1-bit	Arbiter grants bus ownership to DMA. FSM stays in BUS_REQ until this goes high. If CPU is using AHB, DMA waits here.
HADDR	out	32-bit	Byte address on the AHB bus. During read phase: source address. During write phase: destination address. Byte-addressed — every increment of 1 points to the next byte.
HTRANS	out	2-bit	Transfer type. 2'b00=IDLE (no transfer). 2'b10=NONSEQ (new transfer starting). Slaves check this every cycle to decide whether to respond.
HWRITE	out	1-bit	0 = DMA is reading from HADDR (source fetch). 1 = DMA is writing to HADDR (destination store).
HRDATA	in	32-bit	32-bit word returned by the source slave. Valid when HREADY=1. DMA latches this into data_buf on the rising edge of HCLK.
HWDATA	out	32-bit	32-bit word DMA sends to the destination slave. Driven **combinationally** from data_buf. Must be stable when the slave samples it.
HREADY	in	1-bit	The most critical AHB timing signal. 1 = slave has completed, data valid (reads) or captured (writes). 0 = slave needs more time, DMA stalls in current state.

Byte address	Binary	Drop [1:0]	mem index
0x00	0000 0000	00 0000	mem[0]
0x04	0000 0100	00 0001	mem[1]
0x08	0000 1000	00 0010	mem[2]
0x0C	0000 1100	00 0011	mem[3]

4. FSM — State Machine Reference

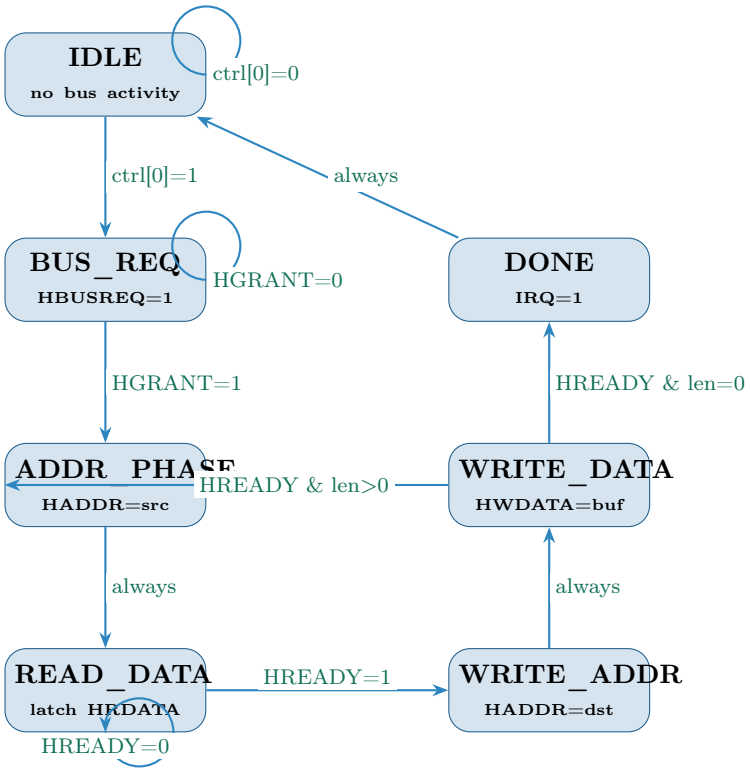
4.1. State Encoding

4.2. State Transition Table

Table 4: Sideband Signals

lightblue **Signal**	**Dir**	**Width**	**Description**
`irq`	out	1-bit	Interrupt to CPU. Asserted when the DMA enters the DONE state (all beats complete). CPU's ISR reads STATUS register, processes the result, and writes INT_CLR to reset the DMA.
`check`	out	1-bit	Debug signal. Asserted when FSM enters WRITE_DATA state. Used during simulation to confirm the write path is being exercised. Not needed in production.

4.3. FSM Diagram



5. Transfer Workflow — Phase by Phase

5.1. Phase 1 — CPU Programs DMA over APB

CPU performs five APB writes. DMA FSM stays in IDLE. No AHB activity.

```
apb_write(32'h00, 32'h0000_0000);  // SRC_ADDR = 0x000 (byte address)
apb_write(32'h04, 32'h0000_0080);  // DST_ADDR = 0x080 (word 32 in mem)
apb_write(32'h08, 32'h0000_0008);  // LEN = 8 words
apb_write(32'h0C, 32'h0000_0001);  // CTRL: EN=1  -> FSM leaves IDLE
```

On the rising PCLK edge where PSEL & PENABLE & PWRITE are all high, the DMA latches the values. Writing `0x1` to CTRL starts everything.

Table 5: DMA Configuration Register Map

lightblue PADDR	Register	Access	Description
0x00	src_addr_reg	R/W	Source byte address. DMA reads data FROM this address. e.g. 0x0000_0000 = start of SRAM.
0x04	dst_addr_reg	R/W	Destination byte address. DMA writes data TO this address. e.g. 0x0000_0080 = word 32 of SRAM.
0x08	len_reg	R/W	Transfer length in number of 32-bit words (beats). e.g. 0x8 = transfer 8 words.
0x0C	ctrl_reg	R/W	Control register. Bit[0]=EN: writing 1 starts the DMA FSM. Other bits reserved for burst, size, direction config.
0x10	status_reg	R only	Status flags. Bit[0]=BUSY, Bit[1]=DONE, Bit[2]=ERR. Written by FSM, read by CPU in ISR. CPU cannot write this.
0x14	INT_CLR	W only	Interrupt clear. CPU writes any value here to clear CTRL[0]=EN, reset all STATUS flags, and return DMA to IDLE.

5.2. Phase 2 — DMA Requests AHB Bus (BUS_REQ)

- DMA asserts HBUSREQ=1 to the bus arbiter.
- DMA drives HTRANS=IDLE — it does **not** yet put any address on the bus.
- If the CPU is using AHB, the DMA waits here. The CPU is unaffected.
- When the arbiter asserts HGRANT=1, FSM transitions to ADDR_PHASE.
- Simultaneously (in IDLE→BUS_REQ transition), DMA copies config register values into working pointers:
  src_ptr ← src_addr_reg, dst_ptr ← dst_addr_reg, len_cnt ← len_reg

5.3. Phase 3 — Read Address Phase (ADDR_PHASE)

- DMA drives: HADDR = src_ptr, HWRITE = 0, HTRANS = NONSEQ.
- The source slave (memory) sees its address and begins preparing the data.
- This state lasts exactly **one clock cycle** — AHB address phase is always one cycle.
- FSM moves to READ_DATA unconditionally.

5.4. Phase 4 — Read Data Phase (READ_DATA)

- Slave drives HRDATA with the word at src_ptr.
- If HREADY=0: slave is not ready (e.g. slow SRAM). DMA holds all signals stable and stays in READ_DATA.
- When HREADY=1: DMA latches data_buf ← HRDATA and increments src_ptr += 4.
- FSM moves to WRITE_ADDR.

Table 6: Internal Working Registers

lightblue **Register**	**Width**	**Purpose**
src_ptr	32-bit	Current source byte address. Initialised from src_addr_reg when CTRL.EN is set. Auto-incremented by 4 (one word = 4 bytes) after each successful read beat.
dst_ptr	32-bit	Current destination byte address. Initialised from dst_addr_reg. Auto-incremented by 4 after each successful write beat.
len_cnt	32-bit	Remaining beat counter. Initialised from len_reg. Decremented by 1 after each complete read+write pair. Transfer ends when len_cnt reaches 0.
data_buf	32-bit	Temporary buffer holding the word read from source until the write to destination completes. Loaded from HRDATA when HREADY=1 in READ_DATA state.
len_cnt_next	32-bit	Combinational (wire). Pre-computed len_cnt - 1, used by the next-state logic to check len_cnt_next == 0 in the same cycle as the decrement — avoiding a 1-beat lag.

5.5. Phase 5 — Write Address Phase (WRITE_ADDR)

- DMA drives: HADDR = dst_ptr, HWRITE = 1, HTRANS = NONSEQ.
- data_buf is now **fully stable** (was loaded last cycle).
- HWDATA = data_buf is already valid combinationally.
- This state lasts one clock cycle. FSM moves to WRITE_DATA.

Why a dedicated WRITE_ADDR state?
AHB is a pipelined bus: address and data are always separated by at least one clock. Without WRITE_ADDR, the write address and data capture of HRDATA happen in the same cycle — the slave sees stale data_buf. Adding WRITE_ADDR gives the pipeline one cycle of separation, ensuring data_buf is stable before the slave captures HWDATA.

5.6. Phase 6 — Write Data Phase (WRITE_DATA)

- HWDATA = data_buf (combinational) is valid for the slave to capture.
- If HREADY=0: slave not ready. DMA stalls.
- When HREADY=1: slave has captured the data.
  - dst_ptr += 4
  - len_cnt = len_cnt_next (pre-computed len_cnt - 1)
  - If len_cnt_next == 0: FSM → DONE
  - If len_cnt_next > 0: FSM → ADDR_PHASE (next beat)

Table 7: FSM State Encoding

lightblue State	Encoding	Meaning
IDLE	3'b000	Waiting. No bus activity. Accepts APB writes.
BUS_REQ	3'b001	DMA asserts HBUSREQ, waits for HGRANT from arbiter.
ADDR_PHASE	3'b010	Drives src_ptr on HADDR, HWRITE=0, HTRANS=NONSEQ.
READ_DATA	3'b011	Waits for HREADY=1; latches HRDATA into data_buf.
WRITE_ADDR	3'b100	Drives dst_ptr on HADDR, HWRITE=1, HTRANS=NONSEQ.
WRITE_DATA	3'b101	HWDATA=data_buf captured by slave; decrements len_cnt.
DONE	3'b110	Asserts IRQ, sets STATUS.DONE. Returns to IDLE next cycle.

5.7. Phase 7 — Transfer Complete (DONE)

- DMA asserts `irq = 1`.
- Sets `STATUS.DONE = 1`, clears `STATUS.BUSY = 0`.
- Drives `HTRANS = IDLE`, lowers `HBUSREQ = 0` — releases bus.
- FSM returns to IDLE next cycle.
- CPU ISR fires, reads STATUS, writes `0x14` (INT_CLR) to reset DMA.

6. Simulation Verification

6.1. Testbench Setup

- **Memory model:** `reg [31:0] mem [0:63]` — 64 words, byte-addressed via `mem[HADDR[7:2]]`.
- **Source:** Words 0–7 pre-filled with `0xCAFE0000 + i`.
- **Destination:** Words 32–39, initialised to `0x00000000`.
- **Arbiter model:** Grants bus after 2 cycles of `HBUSREQ`.
- **Slave model:** Always-ready (`HREADY=1`) unless overridden for wait-state tests.

6.2. Expected vs. Actual Output (After All Fixes)

6.3. Bugs Found and Fixed

Bug 1 — Wrong increment: `jump = 1` should be `JUMP = 4`

Symptom: Duplicate values at destination. Every beat read the same word.
Cause: `src_ptr` incremented by 1 byte each beat. Since memory is indexed as `mem[HADDR[7:2]]`, addresses `0x00, 0x01, 0x02, 0x03` all map to `mem[0]`.
Fix: `localparam JUMP = 32'h4;` — one 32-bit word = 4 bytes.

Table 8: FSM Next-State Transitions

lightblue **Current State**	**Condition**	**Next State**	**Notes**
IDLE	`ctrl_reg[0] == 1`	BUS_REQ	EN bit set by CPU
IDLE	`ctrl_reg[0] == 0`	IDLE	Stay idle
BUS_REQ	`HGRANT == 1`	ADDR_PHASE	Arbiter grants bus
BUS_REQ	`HGRANT == 0`	BUS_REQ	Wait for grant
ADDR_PHASE	always	READ_DATA	Address phase = 1 cycle
READ_DATA	`HREADY == 1`	WRITE_ADDR	Data valid, latched
READ_DATA	`HREADY == 0`	READ_DATA	Slave not ready, stall
WRITE_ADDR	always	WRITE_DATA	Write address = 1 cycle
WRITE_DATA	`HREADY && len_cnt_next == 0`	DONE	Last beat complete
WRITE_DATA	`HREADY && len_cnt_next != 0`	ADDR_PHASE	More beats to go
WRITE_DATA	`HREADY == 0`	WRITE_DATA	Slave not ready, stall
DONE	always	IDLE	1-cycle done pulse

Table 9: Simulation Verification Result — 8-word Transfer

lightblue **Word**	**Source value**	**Expected at dst**	**Result**	**Destination**
0	`0xCAFE0000`	`0xCAFE0000`	**PASS**	`mem[32]`
1	`0xCAFE0001`	`0xCAFE0001`	**PASS**	`mem[33]`
2	`0xCAFE0002`	`0xCAFE0002`	**PASS**	`mem[34]`
3	`0xCAFE0003`	`0xCAFE0003`	**PASS**	`mem[35]`
4	`0xCAFE0004`	`0xCAFE0004`	**PASS**	`mem[36]`
5	`0xCAFE0005`	`0xCAFE0005`	**PASS**	`mem[37]`
6	`0xCAFE0006`	`0xCAFE0006`	**PASS**	`mem[38]`
7	`0xCAFE0007`	`0xCAFE0007`	**PASS**	`mem[39]`

Bug 2 — Read-before-write race on `len_cnt`

Symptom: Transfer stops early or never reaches DONE.
Cause: Next-state logic (`always @(*)`) and the decrement (`len_cnt <= len_cnt - 1`) happen on the same clock edge. Non-blocking assignment means the combinational block sees the *old* value — the `len_cnt == 1` check fires one beat too late.
Fix: `len_cnt_next = len_cnt - 1` as a combinational wire; next-state logic checks `len_cnt_next == 0`.

Bug 3 — AHB pipeline offset: missing WRITE_ADDR state

Symptom: Destination shifted by one word; `mem[32]` always 0, last word never transferred.
Cause: The original code drove the write address (`HADDR=dst_ptr`, `HWRITE=1`) in READ_DATA, the same cycle that `data_buf <= HRDATA` was being loaded. The memory slave captured `HWDATA = data_buf` before `data_buf` had updated — always seeing last beat's data.
Fix: Added dedicated `WRITE_ADDR` state between READ_DATA and WRITE_DATA. Now `data_buf` is fully stable before the write address is driven, and the slave captures correct data in WRITE_DATA.

7. Quick Reference — All Signals at a Glance

Table 10: Complete Signal Reference

lightblue **Signal**	**Dir**	**Width**	**One-line description**
PCLK	in	1	APB clock
HCLK	in	1	AHB clock, drives FSM
PRESETN	in	1	APB async active-low reset
HRESETN	in	1	AHB async active-low reset
PSEL	in	1	APB slave select
PENABLE	in	1	APB 2nd-cycle enable strobe
PWRITE	in	1	APB direction: 1=write, 0=read
PADDR	in	32	APB register offset (bits [7:0] decoded)
PWDATA	in	32	APB write data from CPU
PRDATA	out	32	APB read data to CPU (combinational mux)
PREADY	out	1	APB ready, tied 1 (no stalling)
HGRANT	in	1	Bus grant from AHB arbiter
HBUSREQ	out	1	Bus request to AHB arbiter
HADDR	out	32	AHB byte address (src or dst depending on phase)
HTRANS	out	2	AHB transfer type: 00=IDLE, 10=NONSEQ
HWRITE	out	1	AHB direction: 0=read, 1=write
HRDATA	in	32	AHB read data from slave
HWDATA	out	32	AHB write data to slave (combinational from data_buf)
HREADY	in	1	AHB slave ready: 0=stall, 1=complete
irq	out	1	Interrupt to CPU on transfer completion
check	out	1	Debug: high when in WRITE_DATA state